

KubeQuest — Script de présentation

Script de présentation — KubeQuest

Follow-up du 29 Avril 2026 Document de lecture slide par slide (~2 min par slide, ton oral naturel)

Slide 1 — Titre “KubeQuest”

Bonjour à tous. Aujourd’hui je vais vous présenter le follow-up du projet **KubeQuest**.

Pour ceux qui découvrent le projet, l’idée c’est simple : on a voulu construire de zéro un cluster Kubernetes qui ressemble à ce qu’on trouverait en entreprise. Pas un cluster de jouet pour faire tourner deux pods, mais quelque chose de complet, avec du monitoring, de la sécurité, du déploiement automatique, et une vraie application qui tourne dessus.

L’objectif de ce point d’avancement, c’est de vous montrer où on en est aujourd’hui, ce qu’on a réussi à mettre en place depuis le dernier follow-up, et surtout ce qu’on est capable de démontrer en direct si vous nous le demandez.

Je vais vous emmener pas à pas : on va commencer par l’infra et le choix techno, puis je vais vous montrer tout ce qu’on a déployé, et on finira par les défis qu’on a résolus en chemin. C’est parti.

Slide 2 — Objectif du projet

Alors, l'objectif du projet en une phrase : on voulait déployer un cluster Kubernetes **production-grade** sur AWS.

Concrètement, “production-grade” ça veut dire quoi ? Ça veut dire qu'on ne s'est pas contenté de faire tourner Kubernetes. On a voulu intégrer tout ce qui va autour, et qui rend un cluster réellement utilisable au quotidien.

Il y a quatre piliers qu'on s'est fixés dès le départ :

- Le **monitoring**, pour savoir ce qui se passe en temps réel — métriques, logs, alertes.
- Le **routing**, pour exposer proprement les services vers l'extérieur, via un point d'entrée unique.
- Le **GitOps**, pour que tout ce qui tourne dans le cluster soit décrit dans Git, et que les déploiements se fassent automatiquement.
- Et enfin, une **application réelle** par-dessus, parce qu'un cluster vide ça ne prouve pas grand-chose. Là, on a pris une vraie app Laravel avec sa base de données.

Le fil rouge, c'est de montrer qu'on maîtrise toute la chaîne, depuis la VM AWS brute jusqu'à l'utilisateur qui ouvre l'application dans son navigateur.

Slide 3 — Infrastructure

Côté infrastructure, on a fait simple mais propre. On a quatre VMs sur AWS, toutes en **Amazon Linux 2023**, et toutes en **ARM64** — c'est un détail mais c'est important pour la suite parce que beaucoup d'outils sont pensés pour x86 par défaut.

Chacune des quatre VMs a un rôle bien défini :

- **kube-1**, c'est le cerveau du cluster, le control plane. Mais comme on a peu de machines, il fait aussi worker — c'est-à-dire qu'il peut héberger des applis.
- **kube-2**, c'est un worker pur. Son seul boulot c'est de faire tourner les pods applicatifs.
- **ingress**, c'est une machine **dédiée au point d'entrée web**. Tout le trafic HTTP qui arrive de l'extérieur passe par elle, et personne d'autre ne peut tourner dessus.
- **monitoring**, c'est pareil mais dédiée à l'observabilité. Toute la stack Prometheus, Grafana, Loki est isolée là-dessus.

Pourquoi on isole comme ça ? Parce que si demain l'application explose en charge, on ne veut pas que le monitoring tombe en même temps. Et inversement, si Prometheus se met à manger toute la mémoire, on ne veut pas que ça impacte les requêtes utilisateurs. C'est de la séparation des préoccupations, comme en code.

Slide 4 — Pourquoi K3s ?

Alors là, question légitime : pourquoi on a choisi **K3s** et pas le Kubernetes “officiel” qu’on installe avec kubeadm ?

D’abord, soyons clairs : K3s n’est pas un Kubernetes au rabais. C’est une distribution **100% compatible avec l’API Kubernetes standard**. Tout ce que vous savez faire sur Kubernetes, vous le faites pareil sur K3s — les mêmes commandes kubectl, les mêmes manifests YAML, le même Helm. C’est juste empaqueté différemment.

On l’a choisi pour trois raisons très concrètes :

Premièrement, nos VMs sont en ARM64. K3s supporte ARM nativement, sans bidouille. Kubernetes classique, c’est plus laborieux.

Deuxièmement, on a des ressources limitées. Nos VMs ne sont pas énormes. K3s consomme beaucoup moins de RAM et de CPU que kubeadm — typiquement on parle de quelques centaines de mégas au lieu de plus d’un giga juste pour le control plane. Sur des petites machines, ça change tout.

Troisièmement, et c’est très spécifique à notre contexte : nos VMs s’éteignent chaque soir. Elles sont rallumées le lendemain. Avec K3s, tout est intégré dans des services systemd, donc au redémarrage le cluster repart proprement, tout seul, sans qu’on ait à intervenir. C’est exactement ce qu’on voulait.

En résumé : même puissance fonctionnelle, fraction du poids.

Slide 5 — Ce qu'on a déployé

Voilà la liste de tout ce qui tourne aujourd'hui sur le cluster. Je vais passer rapidement chaque ligne pour que vous voyiez la richesse de ce qui est en place.

nginx-ingress, c'est notre reverse proxy. C'est lui qui reçoit tout le trafic web et qui le redistribue vers le bon service à l'intérieur du cluster. Il tourne en host network sur la machine ingress dédiée.

Prometheus + Grafana, c'est la stack de monitoring. Prometheus collecte toutes les métriques — CPU, RAM, nombre de pods, latence — et Grafana les affiche sous forme de dashboards.

Loki + Promtail, c'est l'équivalent mais pour les logs. Promtail tourne sur chaque nœud, ramasse les logs de tous les pods, et les centralise dans Loki. Du coup on peut chercher dans les logs de toute l'infra depuis une seule interface.

Kubernetes Dashboard, c'est l'interface web officielle pour voir ce qui se passe dans le cluster, en HTTPS avec du contrôle d'accès RBAC.

L'application Laravel, déployée via un **Helm chart custom** qu'on a écrit nous-mêmes, avec replicas, autoscaling, base de données.

ArgoCD, c'est notre brique GitOps. Tout ce qui est dans Git est appliqué automatiquement au cluster.

OPA Gatekeeper, c'est le gendarme du cluster. Il refuse les déploiements qui ne respectent pas nos règles de sécurité.

Sept briques, toutes opérationnelles.

Slide 6 — Features démontrables

Maintenant, parlons concret. Tout ce qu'on a installé, ça sert à quoi ? Voici trois choses qu'on peut démontrer en live, là, maintenant, si vous nous le demandez.

Première démo : l'autoscaling, le fameux HPA. On a un script qui simule une montée en charge sur l'application. Au départ il y a 2 pods qui tournent. Dès que le CPU moyen dépasse 70%, Kubernetes détecte automatiquement qu'il faut plus de puissance, et il fait monter à 3, 4, 5 pods tout seul. Quand la charge redescend, il les retire. C'est complètement automatique, on ne touche à rien.

Deuxième démo : le rollback Helm. On déploie volontairement une version cassée de l'app — une image qui ne démarre pas par exemple. Kubernetes voit que les pods ne deviennent jamais "ready", et nous on lance une commande `helm rollback` qui ramène la version précédente, celle qui marche. En une commande, en quelques secondes. C'est ça la sécurité d'un vrai pipeline de prod : pouvoir revenir en arrière instantanément.

Troisième démo : l'observabilité. J'ouvre Grafana, vous voyez en temps réel le CPU et la RAM de chaque pod, le nombre de requêtes, la latence. Et juste à côté, dans le même Grafana, je peux explorer les logs via Loki — taper une recherche et voir exactement ce qu'a affiché un pod précis à un instant précis.

Trois capacités qui, ensemble, font la différence entre un cluster joujou et un cluster qui tient en production.

Slide 7 — GitOps avec ArgoCD

ArgoCD, c'est probablement la brique la plus importante qu'on a ajoutée. Laissez-moi vous expliquer pourquoi c'est un game changer.

Avant ArgoCD, le workflow classique c'est : un dev modifie un fichier de config, il se connecte au cluster, il tape `kubectl apply`, et voilà. Le problème, c'est qu'on ne sait jamais ce qui tourne réellement dans le cluster, ni qui a appliqué quoi, ni quand. C'est le bazar.

Avec ArgoCD, on inverse la logique : **la source de vérité, c'est Git**. Tout ce qu'on veut voir tourner dans le cluster est décrit dans notre repo. ArgoCD surveille le repo en permanence, et dès qu'on fait un `git push`, il détecte le changement et l'applique automatiquement au cluster.

Concrètement, on a structuré ça en **une AppProject "KubeQuest" et trois Applications** : une pour l'infrastructure, une pour l'environnement staging, une pour la production.

La structure des fichiers utilise **Kustomize** : on a un dossier `base` avec la config commune, et des `overlays` par environnement qui surchargent juste ce qui change entre staging et production. Ça évite la duplication.

L'interface d'ArgoCD est exposée via notre Ingress. On a eu un petit souci au passage qu'on a corrigé : par défaut ArgoCD veut du HTTPS entre l'Ingress et lui, ce qui crée une double couche TLS et fait des erreurs 502. On a désactivé le HTTPS interne, et tout fonctionne.

Le bénéfice numéro un : **rollback en `git revert`**. Une mauvaise modif ? Un commit en arrière et ArgoCD ramène l'état précédent automatiquement.

Slide 8 — Principaux défis résolus

Bien sûr, ça ne s'est pas fait sans frictions. Je voulais vous partager quatre obstacles qu'on a rencontrés et résolus, parce que ce sont souvent ces détails-là qui font la vraie différence en projet d'infra.

Premier défi : l'architecture ARM64. Plein d'outils CLI qu'on installe, par défaut, téléchargent la version x86. On a écrit nos scripts pour qu'ils détectent automatiquement l'architecture avec `uname -m` et qu'ils récupèrent le bon binaire. Sans ça, on se retrouvait avec des binaires qui ne s'exécutaient même pas.

Deuxième défi : un conflit dans Grafana. Quand on a branché Prometheus et Loki en même temps comme datasources, on s'est retrouvé avec deux sources marquées `isDefault: true`. Grafana plantait au démarrage. On a corrigé en patchant directement le ConfigMap pour n'en garder qu'une seule par défaut.

Troisième défi : le Kubernetes Dashboard version 7. La nouvelle version exige obligatoirement du HTTPS pour les cookies CSRF — sinon la connexion est refusée. Comme on n'avait pas encore mis en place cert-manager, on a généré un certificat TLS auto-signé pour débloquer la situation.

Quatrième défi : le chart Helm officiel du Dashboard a été retiré du Helm repo classique. On ne pouvait plus l'installer comme avant. On l'a téléchargé directement depuis les GitHub Releases et on l'a embarqué dans notre repo. Au passage, ça nous rend complètement autonomes — pas besoin d'accès Internet au moment du déploiement.

Quatre petits obstacles, mais c'est exactement ce genre de choses qu'on rencontre en vrai sur un projet d'infra.

Slide 9 — Sécurité

On termine avec la sécurité, parce qu'un cluster en production sans sécurité, ça n'existe pas.

On a mis en place **OPA Gatekeeper**, qui est un système de politiques. En gros, c'est un gendarme qui regarde tout ce qu'on essaie de déployer dans le cluster, et qui refuse ce qui ne respecte pas les règles. On a défini trois contraintes :

Première contrainte : no-privileged. On interdit les conteneurs qui tournent en mode "privilegié", c'est-à-dire qui ont quasiment les mêmes droits que la machine hôte. Si quelqu'un essaie de déployer un pod privilégié, Gatekeeper refuse le manifest, point.

Deuxième contrainte : required-labels. On impose que chaque ressource du cluster ait au moins trois labels : `app`, `env`, et `owner`. Pourquoi ? Parce que sans labels, on ne sait jamais à qui appartient quoi, ni dans quel environnement on est. C'est de l'hygiène opérationnelle.

Troisième contrainte : required-resources. Chaque pod doit déclarer ses requests et limits CPU et mémoire. Sans ça, un pod peut consommer toutes les ressources du nœud et étouffer les autres. C'est le genre de règle qui évite les incidents de prod.

À côté de Gatekeeper, on a préparé **Dex + oauth2-proxy** pour ajouter du SSO sur les interfaces internes — Grafana, le Dashboard, ArgoCD. L'idée c'est qu'au lieu d'avoir un mot de passe par outil, on se connecte une fois via un provider OIDC et on accède à tout. C'est préparé, pas encore branché en production, mais c'est la prochaine étape.

Dernier point : aujourd'hui les secrets sont gérés via les **Kubernetes Secrets** standards, qui sont juste encodés en base64 — pas chiffrés. À terme, on veut passer à **Sealed Secrets** ou **External Secrets** pour pouvoir versionner les secrets dans Git sans risque. C'est dans la roadmap.

Voilà, merci pour votre attention. Je suis prêt à répondre à vos questions et à faire des démos en direct sur ce que vous voulez voir.